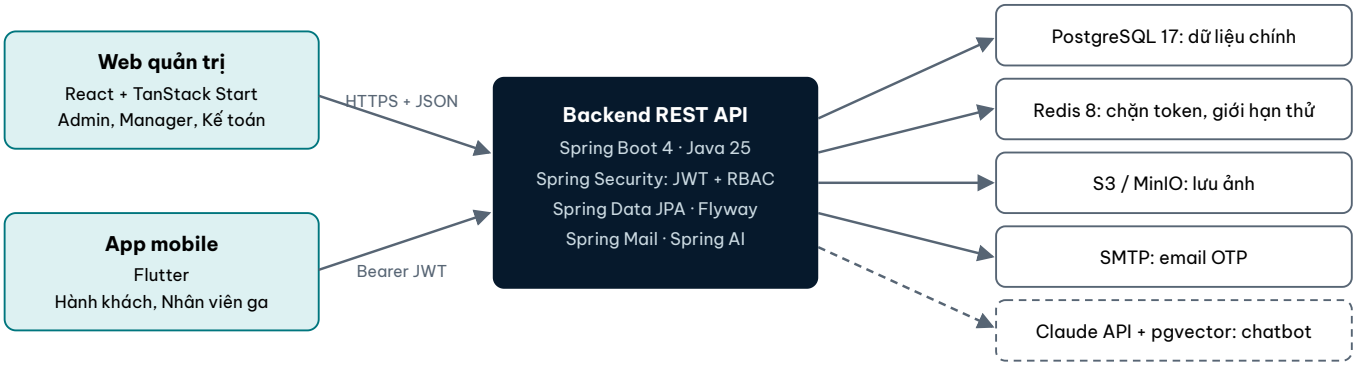
HopOn: Tổng hợp công nghệ

Hệ thống đặt vé Metro TP.HCM · Web quản trị, app mobile và backend · Cập nhật 04/11/2026

● Đang chạy trong code ● Đã có một phần, chưa nối đủ ○ Đề xuất, chưa có code

1. Kiến trúc tổng thể



Web và mobile dùng chung một backend qua REST, không truy cập trực tiếp cơ sở dữ liệu. Mọi quyết định về quyền và nghiệp vụ phải kiểm tra ở backend; giao diện chỉ ẩn hoặc hiện chức năng.

> **Hiện trạng API:** backend đã có xác thực, hồ sơ, tài khoản, vai trò và quyền, thông báo, nhật ký bảo mật, banner. Các nghiệp vụ vé, ga, tuyến, giao dịch, Hành trình xanh, sự cố và xác nhận sinh viên chưa có API, nên web và mobile đang dùng dữ liệu giả lập.

2. Frontend web

Công nghệ	Vai trò
● React 19, TypeScript 5.8	Dựng giao diện theo component, kiểm tra kiểu khi build.
● TanStack Start, Router	Định tuyến theo file, render phía server, tách code từng trang. Build ra server Node bằng Nitro.
● TanStack Query	Gọi API, cache và làm mới dữ liệu máy chủ, ví dụ chuông thông báo.
● Tailwind CSS 4, Radix UI	Hệ thống màu và giao diện sáng/tối, bộ component theo kiểu shadcn/ui, hỗ trợ bàn phím.
● React Hook Form, Zod	Quản lý form và kiểm tra dữ liệu nhập.
● Recharts, Leaflet, SVG	Biểu đồ thống kê, bản đồ ga và tuyến, mô hình 3D tự dựng bằng SVG nên không tốn thư viện WebGL.
● ExcelJS	Xuất báo cáo `.xlsx`; chỉ tải khi người dùng bấm xuất để trang không nặng.
● Vite 8, Vitest, ESLint	Công cụ build, kiểm thử tự động, kiểm tra chất lượng code.

Cách hoạt động: `lib/api/client.ts` gắn access token vào mỗi request; khi gặp 401 thì tự làm mới token một lần, dùng Web Locks để nhiều tab không cùng làm mới. Khi deploy, file tĩnh được nén sẵn gzip và brotli lúc build (trang nặng khoảng 1,1 MB còn khoảng 290 KB tải về) và cache lâu dài theo tên file có mã băm.

3. App mobile

Công nghệ	Vai trò
● Flutter, Dart	Một codebase cho Android và iOS; một app chung cho hành khách và nhân viên ga, giao diện đổi theo vai trò.
● Riverpod, go_router	Quản lý trạng thái và tiêm phụ thuộc; điều hướng có chặn theo đăng nhập và vai trò.
● Dio, Secure Storage	Gọi REST, tự làm mới token; lưu token trong kho bảo mật của thiết bị.
● mobile_scanner, qr_flutter	Nhân viên quét QR khi check-in, check-out; hành khách hiển thị QR vé.
● flutter_map, geolocator	Bản đồ mạng lưới và tìm ga gần nhất.
● stomp_dart_client	Nghe vị trí tàu qua WebSocket/STOMP. App đã có client; backend chưa có endpoint `/ws`.

Cách hoạt động: mỗi tính năng chia theo Clean Architecture thành `domain`, `data`, `presentation`. Mỗi nguồn dữ liệu có bản thật và bản giả lập, chuyển bằng cấu hình `Env.useFakeData`, nên giao diện không phải sửa khi backend hoàn thiện.

4. Backend

Công nghệ	Vai trò
● Spring Boot 4.1, Java 25	Nền tảng REST API, chạy trong Docker cùng các dịch vụ khác qua Docker Compose.
● Spring Security, JWT	Xác thực không lưu phiên. Access token hạn 15 phút, refresh token hạn 14 ngày.
● PostgreSQL 17, JPA, Flyway	Lưu dữ liệu quan hệ; Flyway quản lý thay đổi cấu trúc bảng theo phiên bản, hiện có 9 migration.
● Redis 8	Danh sách access token đã thu hồi, đăng ký đang chờ xác thực, đếm số lần đăng nhập và nhập OTP sai.
● Spring Mail, Thymeleaf	Gửi email OTP theo mẫu; dùng Mailpit khi phát triển. OTP hạn 10 phút, sai quá 5 lần thì hủy.
● AWS SDK S3, MinIO	Lưu ảnh banner, xem mục 6.
● Testcontainers, GreenMail	Kiểm thử tích hợp với PostgreSQL và máy chủ mail thật trong container.
● Spring AI 2.0	Đã khai báo starter Anthropic; chưa có code gọi mô hình, xem mục 7.

5. RBAC: phân quyền theo vai trò

RBAC (Role-Based Access Control) gán quyền cho vai trò, rồi gán vai trò cho người dùng, thay vì gán quyền cho từng người.

Dữ liệu: `app_user` ↔ `user_role` ↔ `role` ↔ `role_permission` ↔ `permission`. Có sẵn 10 mã quyền như `TICKET_VALIDATE`, `USER_MANAGE`, `RBAC_MANAGE`.

> **Luồng một request:** (1) Đăng nhập, JWT chứa claim `roles`. (2) Client gửi `Authorization: Bearer`. (3) Spring Security xác minh chữ ký, `RoleAuthoritiesConverter` đổi vai trò thành quyền bằng `RolePermissionCache` trong bộ nhớ, không truy vấn DB mỗi request. (4) API kiểm tra bằng `@PreAuthorize("hasAuthority('USER_MANAGE')")`. Thiếu token trả 401, thiếu quyền trả 403.

- **Linh hoạt:** kiểm tra theo mã quyền, nên Admin tạo vai trò mới và gán quyền là dùng được ngay. Cache nạp lại khi đổi quyền và định kỳ 5 phút.
- **Vai trò:** backend có sẵn PASSENGER, STAFF, ADMIN. ● MANAGER và ACCOUNTANT được tạo qua trang Vai trò trên web, chưa có sẵn trong dữ liệu khởi tạo.
- **Trên web:** `lib/auth/permissions.ts` khai báo một nơi duy nhất vai trò nào mở được trang và thao tác nào; menu, chặn đường dẫn và test đều đọc từ đây.
- ● **Luồng Admin đề xuất, Manager duyệt** hiện nằm ở web với dữ liệu giả lập; khi làm thật, backend phải lưu yêu cầu duyệt và tự kiểm tra lại quyền.

6. S3 và MinIO: lưu trữ đối tượng

S3 lưu file theo cặp `bucket/key`, không lưu trong cơ sở dữ liệu, nên database nhẹ và mở rộng dễ. MinIO là dịch vụ tương thích giao thức S3, dùng để chạy trên máy cá nhân; đổi sang Amazon S3 chỉ cần đổi cấu hình `S3_ENDPOINT` và khóa truy cập, không phải sửa code.

● **Trong dự án:** interface `ObjectStorage` có các thao tác ghi, đọc; bản cài đặt `S3ObjectStorage` chỉ bật khi `S3_ENABLED=true`. Ảnh banner và file `manifest.json` nằm trên bucket, app tải qua `GET /api/content/banners/{id}/image`, Admin tải lên và xóa qua API có kiểm tra quyền. Storage lỗi thì trả danh sách rỗng để app dùng ảnh có sẵn, Trang chủ không bị hỏng.

○ **Nên dùng tiếp cho:** ảnh sự cố và ảnh thẻ sinh viên. Đây là dữ liệu nhạy cảm nên để bucket riêng tư, chỉ cấp link có hạn (presigned URL), và tự xóa ảnh thẻ sau khi có kết quả xác nhận.

7. AI cho chatbot và RAG

● **Hiện trạng:** app đã có giao diện chat và gọi `POST /api/chat`, web có trang kho FAQ, backend đã khai báo Spring AI nhưng chưa có API chat. Phần dưới là phương án đề xuất; chưa kiểm thử với dữ liệu thật.

Mô hình có thể dùng

Mô hình	Giá (USD/1 triệu token)	Dùng cho
Claude Haiku 5.5	0,10 vào · 0,50 ra	Phân loại câu hỏi, trả lời FAQ ngắn. Rẻ, phù hợp lượng hỏi lớn.
Claude Sonnet 5.5	2 vào · 10 ra	**Đề xuất chính:** trả lời RAG, gọi công cụ tra vé và điểm, đọc ảnh thẻ sinh viên.
Claude Opus 5.5	4 vào · 20 ra	Câu hỏi nhiều bước khó. Chatbot tra cứu thường không cần mức này.
Embedding	Theo nhà cung cấp	Claude không có API embedding. Có thể dùng Voyage AI hoặc BGE-M3 mã nguồn mở tự chạy; cần thử với tiếng Việt trước khi chốt.

Giá Claude theo bảng công bố của Anthropic, cập nhật 06/10/2026; Haiku tính cho prompt đến 100K token. Spring AI cũng hỗ trợ OpenAI, Google Gemini và mô hình tự chạy, nên có thể đổi nhà cung cấp mà ít phải sửa code.

RAG hoạt động thế nào

RAG (Retrieval-Augmented Generation) tìm tài liệu liên quan trước, rồi mới yêu cầu mô hình trả lời dựa trên đúng tài liệu đó, nhờ vậy giảm trả lời sai và cập nhật kiến thức không cần huấn luyện lại.

1. **Nạp dữ liệu:** lấy FAQ, quy định vé, thông tin ga và tiện ích, chia thành đoạn ngắn, tạo embedding, lưu vào `pgvector` ngay trên PostgreSQL hiện có (đổi image `postgres:17-alpine` sang bản có sẵn pgvector).
2. **Tìm kiếm:** tạo embedding cho câu hỏi, lấy các đoạn gần nhất, kết hợp tìm từ khóa để bắt đúng tên ga và mã vé; có thể xếp hạng lại bằng reranker.
3. **Sinh câu trả lời:** đưa các đoạn tìm được vào prompt, yêu cầu chỉ trả lời theo nội dung đó và nói rõ khi không biết. Dùng prompt caching cho phần hướng dẫn cố định để giảm chi phí.
4. **Dữ liệu cá nhân dùng công cụ, không đưa vào RAG:** vé và CPoint của khách được lấy bằng tool calling, lọc theo người dùng trong JWT, để không lộ dữ liệu của người khác.
5. **Học từ thực tế:** câu hỏi chưa trả lời được lưu vào `chatbot_message`; Admin bổ sung vào kho FAQ, hệ thống tạo lại embedding.

Trên Spring AI: dùng `ChatClient`, `PgVectorStore` và tool calling. Khóa API chỉ nằm ở backend; nội dung tài liệu và lời nhắn người dùng phải coi là dữ liệu, không phải lệnh, để hạn chế prompt injection.